

How to Set a Replication Slot — Hands-on Lab

You set a slot by creating it on the current primary. Patroni already sets physical slots for acid-postgres-ha-1 and -2. This lab sets one extra logical slot named lab_slot, proves it exists, then drops it.

Do not pg_drop_replication_slot any physical slot that belongs to a cluster member. Do not create extra physical slots for -1/-2 on this operator cluster.

Step 0 — Work only on the primary

```
NS=postgres-ha
PRIMARY=$(for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  v=$(kubectl -n "$NS" exec "$p" -- psql -U postgres -Atqc "select pg_is_in_recovery();")
  if [ "$v" = "f" ]; then echo "$p"; break; fi
done)
echo "PRIMARY=$PRIMARY"
```

CREATE/DROP SLOT on a replica fails or is the wrong place. Slots live on the primary.

Step 1 — Snapshot slots Patroni already set

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "
SELECT slot_name, slot_type, active, restart_lsn
FROM pg_replication_slots
ORDER BY slot_type, slot_name;
"
```

Write down existing names. You will only add lab_slot. Everything already listed stays untouched.

Step 2 — Capacity and wal_level

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "
SHOW wal_level;
SHOW max_replication_slots;
SHOW max_wal_senders;
SELECT count(*) AS slots_in_use FROM pg_replication_slots;
"
```

Logical slot requires wal_level=logical. If it is replica, stop this lab and set wal_level in the Zalando/Patroni CR, then roll pods. slots_in_use must be less than max_replication_slots.

Step 3 — Set (create) the logical slot

This is the actual SET step: pg_create_logical_replication_slot on the primary.

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "
SELECT pg_create_logical_replication_slot('lab_slot', 'pgoutput');
"
```

Arguments: slot name lab_slot, plugin pgoutput (same plugin CREATE SUBSCRIPTION uses).

If you see 'replication slot already exists', either it was left from an earlier run or a subscription used that name. List slots and drop only lab_slot if it is yours.

Step 4 — Confirm it was set

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "
SELECT slot_name, slot_type, plugin, database, active,
       confirmed_flush_lsn, restart_lsn
FROM pg_replication_slots
WHERE slot_name = 'lab_slot';
"
```

Expect slot_type=logical, plugin=pgoutput, active=f (no subscriber connected yet). active=f is OK for a few minutes only.

Step 5 — Optional: attach a subscription to this slot

Skip if acid-recovery-test-0 is not Ready. Otherwise the subscriber uses the slot you set instead of creating a new one.

```
# publisher must have a publication first (example)
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c "
CREATE TABLE IF NOT EXISTS public.logical_demo (
```

```

    id int PRIMARY KEY, note text);
CREATE PUBLICATION IF NOT EXISTS app_pub FOR TABLE public.logical_demo;
"

# subscriber -- create_slot = false, slot_name = lab_slot
kubectl -n "SNS" exec acid-recovery-test-0 -- psql -U postgres -d recoverydb -c "
CREATE TABLE IF NOT EXISTS public.logical_demo (
    id int PRIMARY KEY, note text);
DROP SUBSCRIPTION IF EXISTS app_sub;
CREATE SUBSCRIPTION app_sub
    CONNECTION 'host=acid-postgres-ha.postgres-ha.svc.cluster.local port=5432 user=postgres dbname=banking'
    PUBLICATION app_pub
    WITH (create_slot = false, slot_name = 'lab_slot', copy_data = true);
"

```

After this, `lab_slot` should show `active=t` on the primary.

Step 6 — Unset (drop) what you created

If you created `app_sub`, drop the subscription first so it releases the slot.

```

kubectl -n "SNS" exec acid-recovery-test-0 -- psql -U postgres -d recoverydb -c "DROP SUBSCRIPTION IF EXISTS app_sub;" 2>/dev/n

kubectl -n "SNS" exec "$PRIMARY" -- psql -U postgres -c "
SELECT pg_drop_replication_slot('lab_slot');
"

```

If drop says slot is active, the subscription is still running. DROP SUBSCRIPTION first.

Step 7 — Verify only Patroni slots remain

```

kubectl -n "SNS" exec "$PRIMARY" -- psql -U postgres -c "
SELECT slot_name, slot_type, active
FROM pg_replication_slots
ORDER BY 1;
"

```

`lab_slot` must be absent. Physical member slots must still be present and preferably `active=t`.

What you are not setting in this lab

Physical slots for standbys — Patroni/operator already set them (`use_slots`). Permanent failover-safe logical slots — Patroni DCS slots: YAML, not this SQL drill. `max_replication_slots` — cluster parameter in the postgresql CR.

Command cheat sheet

Action	SQL on primary
Set logical slot	<code>SELECT pg_create_logical_replication_slot('lab_slot', 'pgoutput');</code>
Set physical slot (do not on this HA cluster)	<code>SELECT pg_create_physical_replication_slot('name');</code>
List	<code>SELECT * FROM pg_replication_slots;</code>
Unset lab slot	<code>SELECT pg_drop_replication_slot('lab_slot');</code>
Slot via subscription	<code>CREATE SUBSCRIPTION ... WITH (create_slot = true);</code>
Use a slot you set	<code>CREATE SUBSCRIPTION ... WITH (create_slot = false, slot_name = 'lab_slot');</code>